



浙江大学
ZHEJIANG UNIVERSITY



循环神经网络

陈建海 计算机学院

2024年11月25日



课程二维码

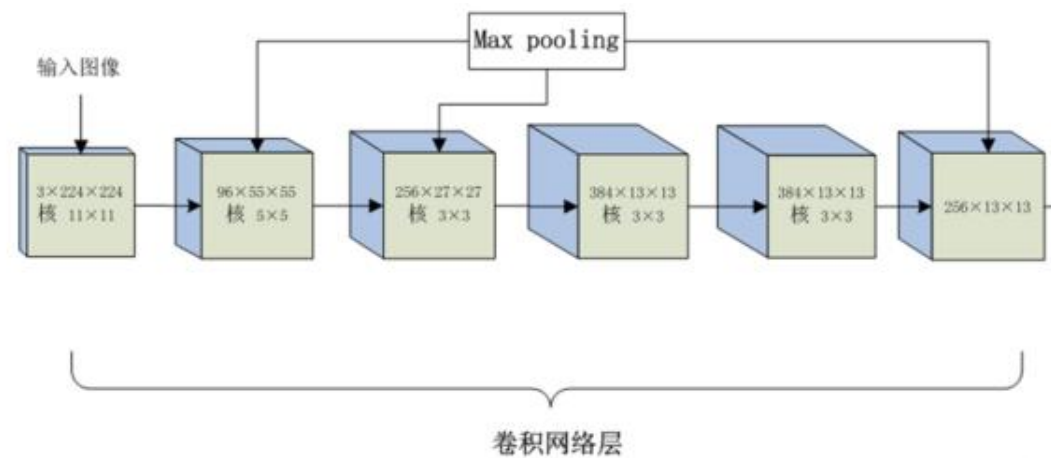
本课件仅供教学使用，未经许可不得转载



AlexNet



第一部分：以下为卷积网络层						
层数	算法	kernels	kernel_size	padding	stride	output_size
1	卷积	96	11×11	[1,2]	4	[55,55,96]
	激活函数	ReLU				
	最大池化		3×3	0	2	[27,27,96]
2	卷积	256	5×5	[2,2]	1	[27,27,256]
	激活函数	ReLU				
	最大池化		3×3	0	2	[13,13,256]
3	卷积	384	3	[1,1]	1	[13,13,384]
	激活函数	ReLU				
4	卷积	384	3	[1,1]	1	[13,13,384]
	激活函数	ReLU				
5	卷积	256	3	[1,1]	1	[13,13,256]
	激活函数	ReLU				
	最大池化		3×3	0	2	[6,6,256]



例9-2：对如下的特征图进行平均池化计算，池化窗口2×2，

23	34	55	32	66	43
15	43	27	30	39	54
33	67	47	28	36	49
56	89	90	35	77	65
54	32	37	48	65	32
63	67	38	43	29	54

步长=2的池化

提纲

- 1 RNN
- 2 LSTM
- 3 股票走势预测
- 4 案例分析

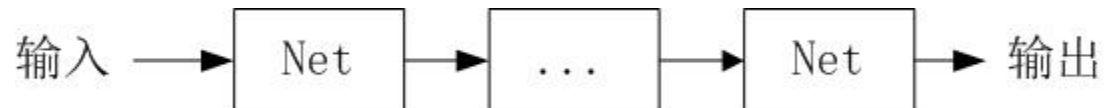


RNN

三种神经网络



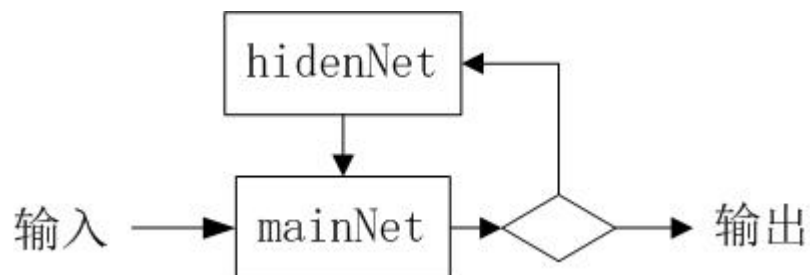
前馈神经网络



多层感知机 MLP

卷积神经网络 CNN

反馈神经网络



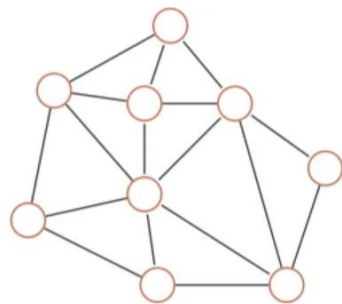
循环神经网络 RNN

长短程记忆网络 LSTM

Hopfield网络

波尔兹曼机

图网络



知识图谱

社交网络

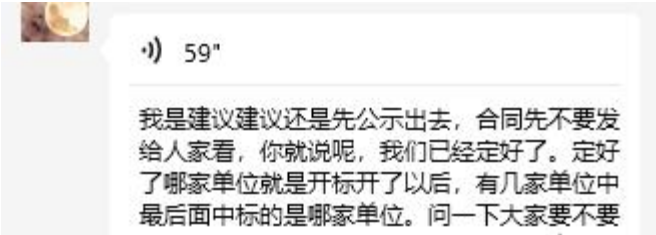
城市交通

序列数据处理任务



自然语言处理：（单词预测）人之初，性本善，性相（近），习相远
翻译：我在学人工智能 → I'm learning artificial intelligence
语义理解：“我的手机昨天丢了，你能帮我买128G的苹果吗？”

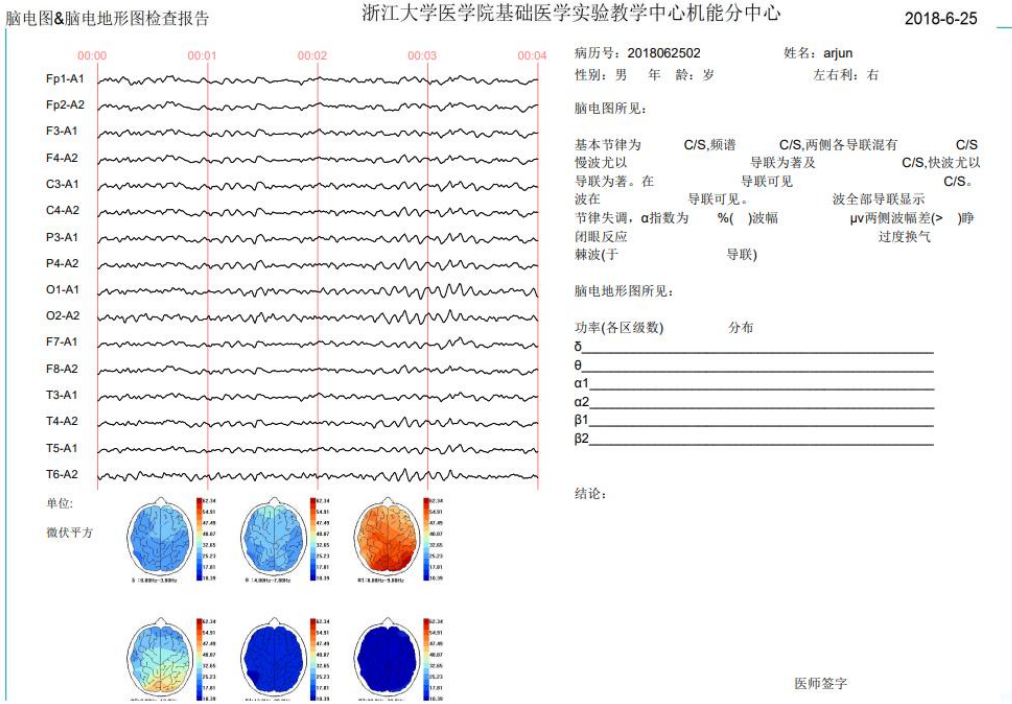
语音识别：

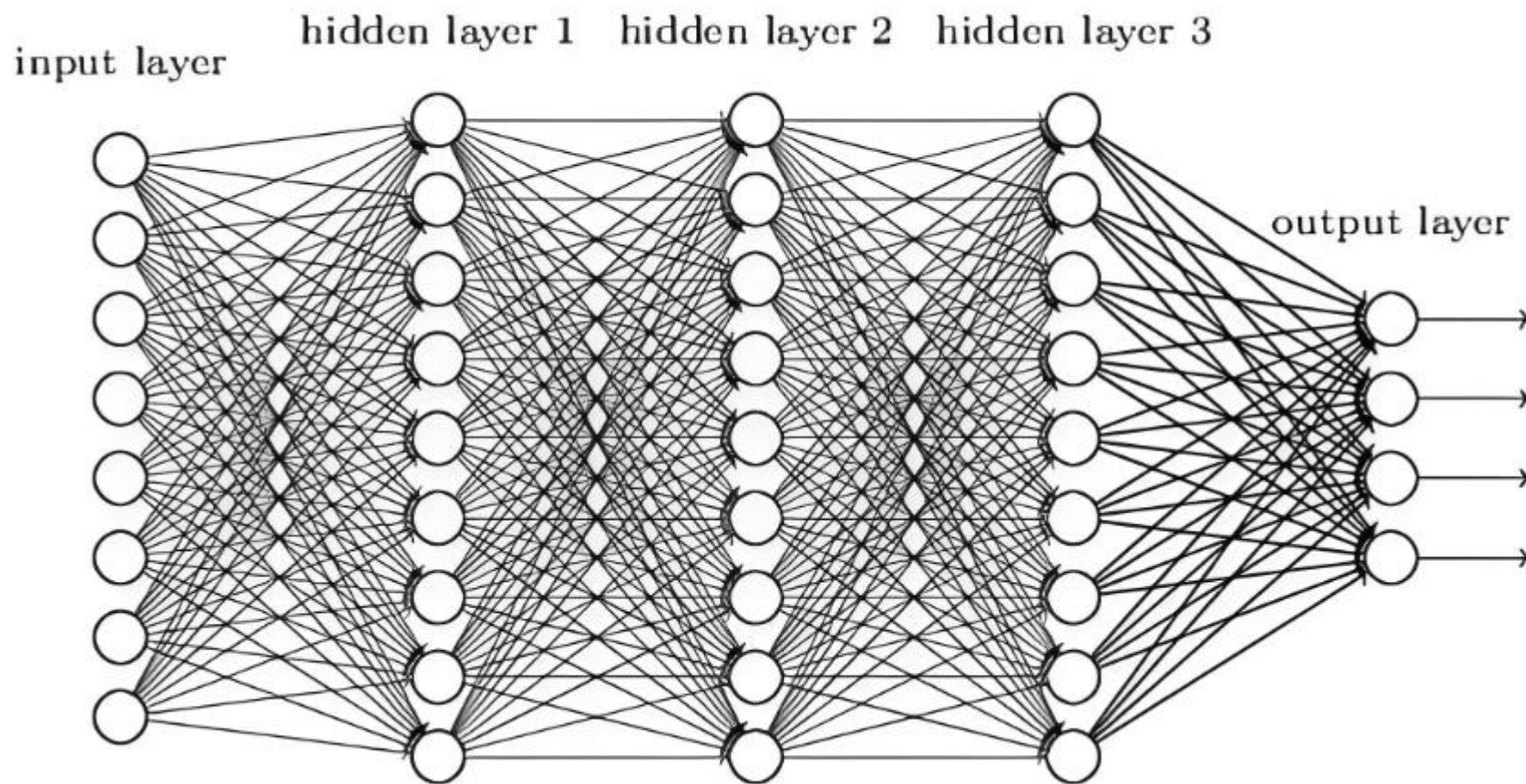


股市预测：

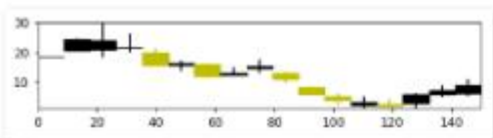


生物信号处理：

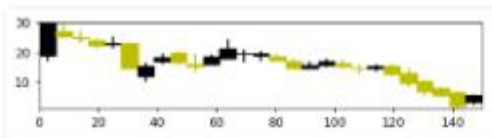




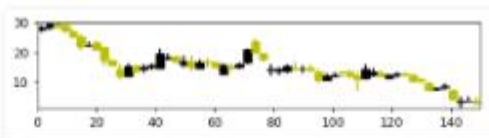
假如用MLP对序列数据进行建模



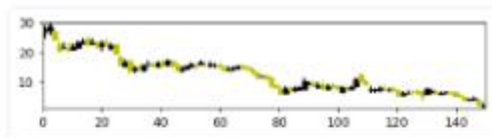
d0001.png



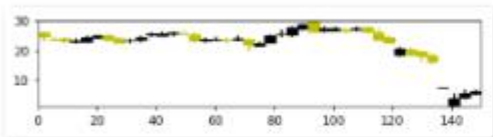
d0002.png



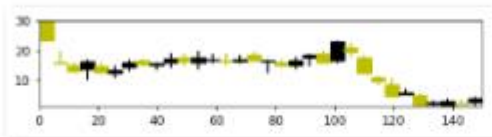
d0003.png



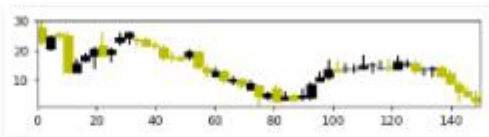
d0004.png



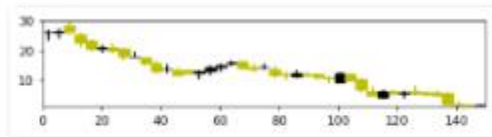
d0007.png



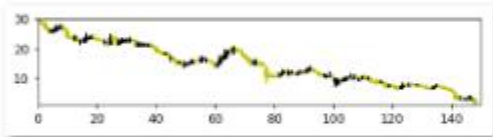
d0008.png



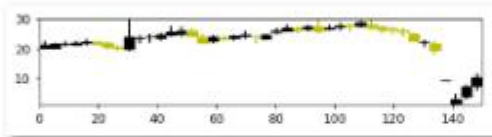
d0009.png



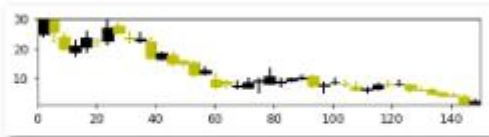
d0010.png



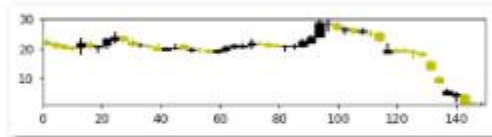
d0013.png



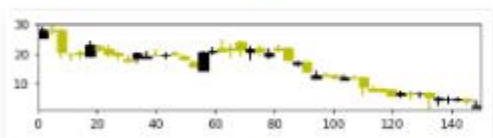
d0014.png



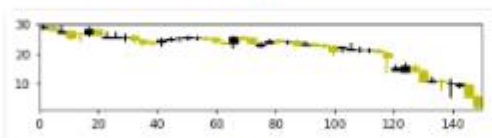
d0015.png



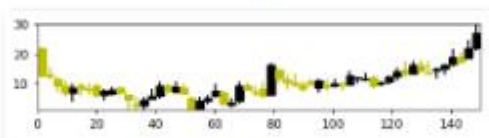
d0016.png



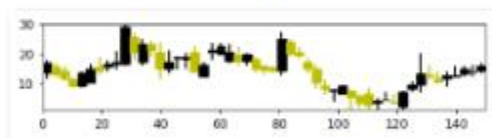
d0019.png



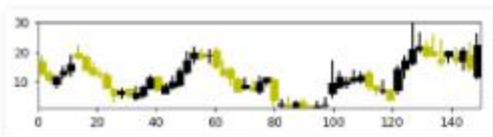
d0020.png



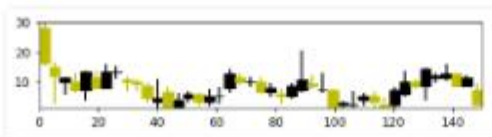
p0001.png



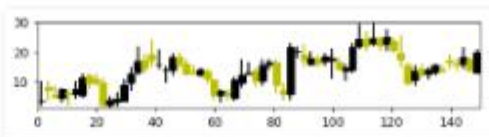
p0002.png



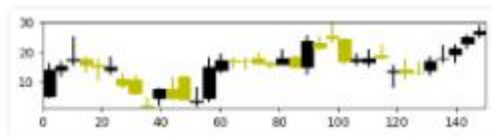
p0005.png



p0006.png



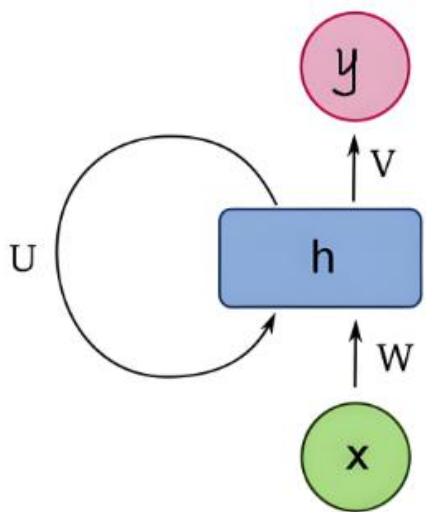
p0007.png



p0008.png

假如用CNN对序列数据进行建模

- **循环神经网络 (Recurrent Neural Network, RNN)**：是一种特殊类型的反馈神经网络，专门用于处理序列数据。
- 核心思想：循环结构，网络能捕捉和利用序列中的顺序依赖性信息。



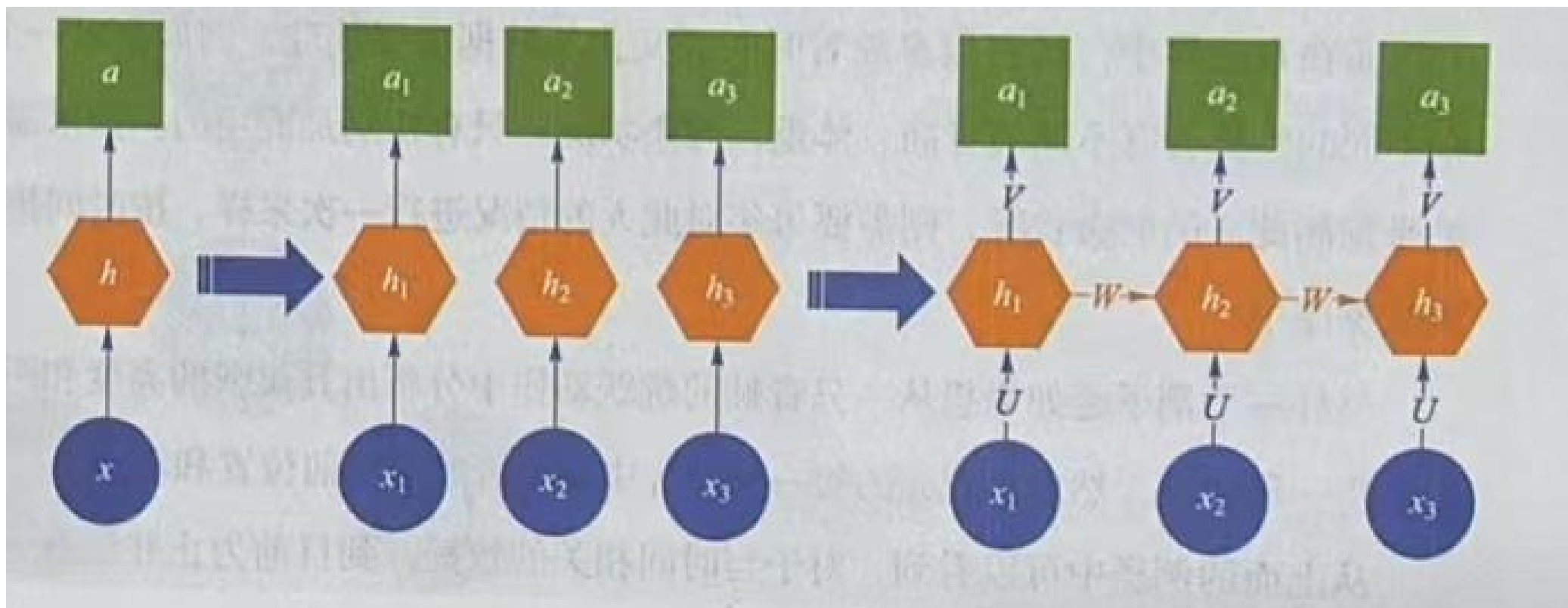
表示法一

- RNN的基本单元是一个具有循环连接的神经网络层——允许网络在处理每个时间步的数据时，能**利用之前**时间步信息。
- 具体地，RNN在每个时间步都会**接收一个输入**，并**产生一个输出**，同时其内部状态（隐藏状态）会被更新并传递到下一个时间步。

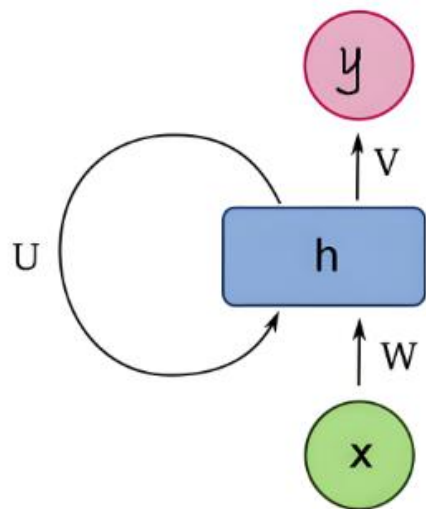


- 1933年，西班牙神经生物学家Rafael Lorenre de No 发心大脑皮层的解剖结构允许刺激在神经回路中循环传递，提出**回荡回路假设**
- 1982年，美国学者John Hopfield实用二院阶段建立了具有结合存储能力的神经网络，即**Hopfield神经网络**
- 1986年，Michael I.Jordan 基于Hopfield建立了新的循环神经网络，**Jordan网络**
- 1990年，Jeffrey Elman提出了第一个全连接的循环神经网络，**简单循环神经网络：SRN**
- 1990年，Paul Werbos提出了循环神经网络的随时间反向传播，BPTT。
- 1991年，Sepp Hochreiter发现了循环神经网络的长期以来问题，引入大量优化，于是产生了**神经历史压缩器NHC、长短期记忆网络LSTM、门控循环单元网络GRU、回声状态网络、独立循环神经网络**等。

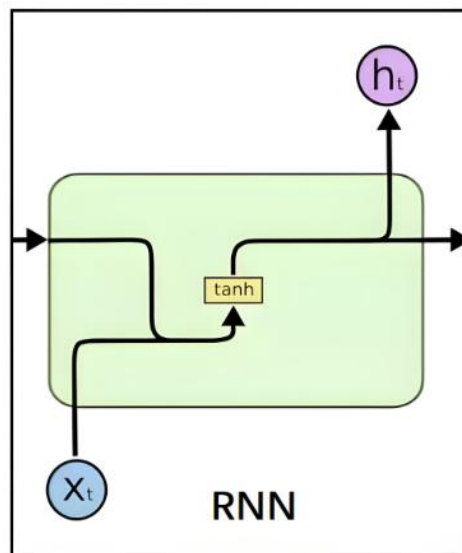
从前馈神经网络到RNN



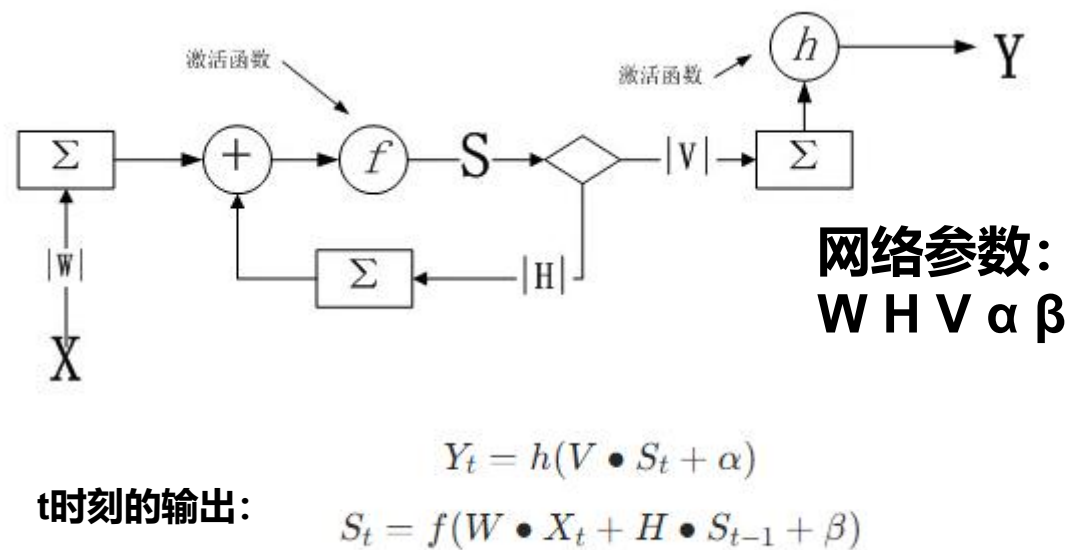
RNN的结构



表示法一



表示法二

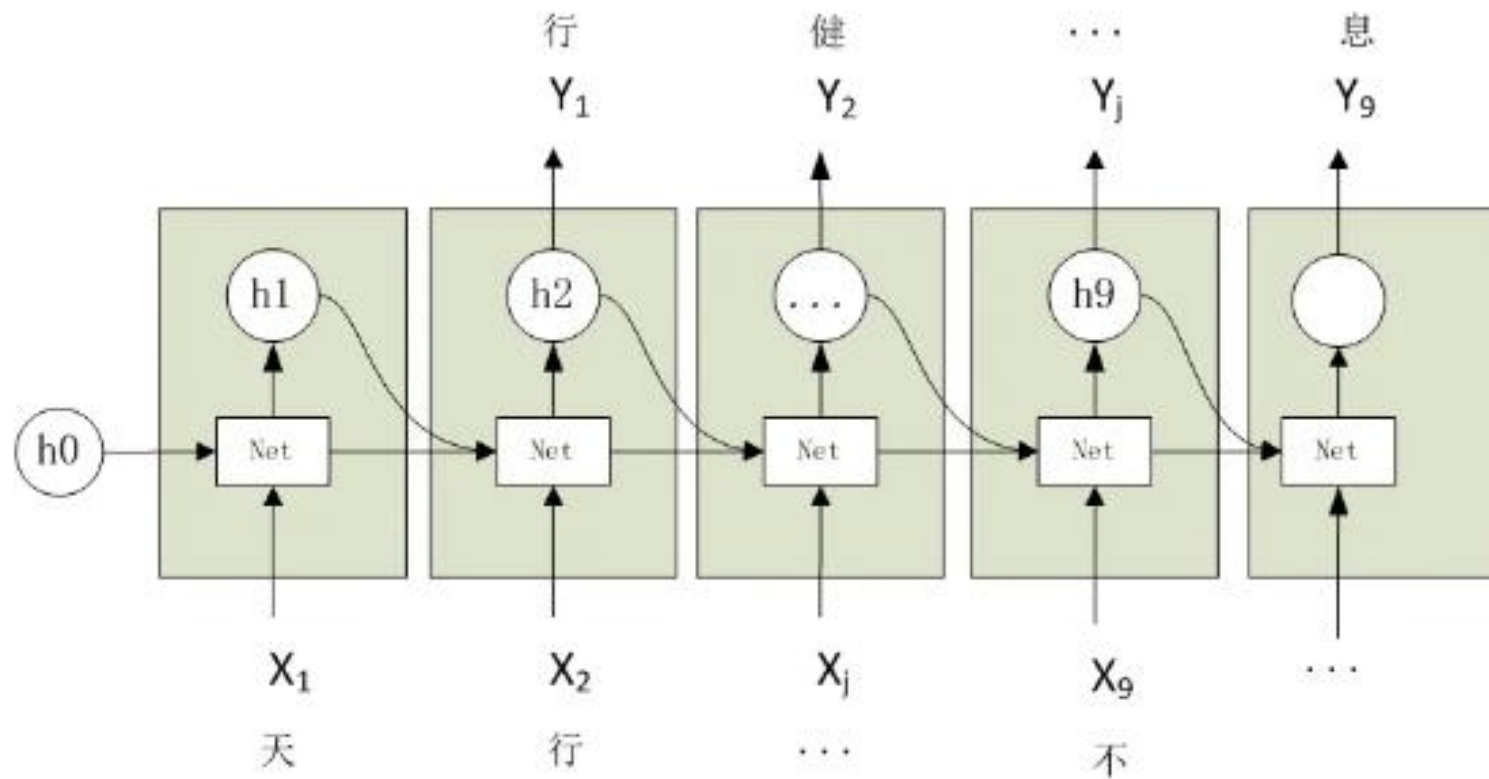


表示法三

RNN按序列展开



预测：“天行健，君子以自强不息（）”



数据集的构建

$i =$	1	2	3	4	5	6	7	8	9
x_i	天	行	健	君	子	以	自	强	不
y_i	行	健	君	子	以	自	强	不	息

——→词向量

——→词向量

	水果	公司	手机	粮食
$V_{\text{橘子}}$	0.92	0.23	0.02	0.19
$V_{\text{香蕉}}$	0.95	0.13	0.09	0.25
$V_{\text{苹果}}$	0.96	0.77	0.85	0.15
$V_{\text{小米}}$	0.08	0.81	0.98	0.87
$V_{\text{华为}}$	0.02	0.93	0.95	0.01

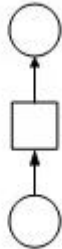
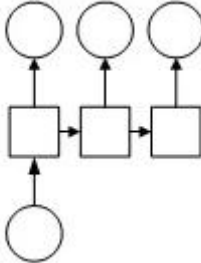
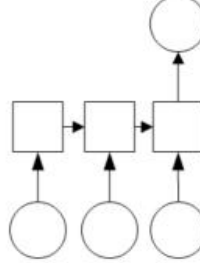
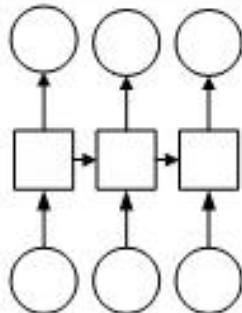
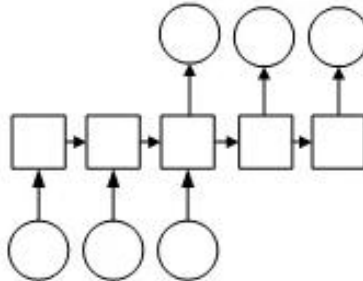
词义特征向量

黄	[1,0,0,0,0]
山	[0,1,0,0,0]
落	[0,0,1,0,0]
叶	[0,0,0,1,0]
松	[0,0,0,0,1]
叶	[0,0,0,1,0]
落	[0,0,1,0,0]
山	[0,1,0,0,0]
黄	[1,0,0,0,0]

词义相关性

RNN分类



— 对 —		图像去噪 数据加密	— 对 多		关键词→写作 拼写纠正	多 对 —		语句情感分类 垃圾邮件识别 股票涨跌判断 关键词提取
同 步 多 对 多		命名实体识别 词性标注	异 步 多 对 多		语句预测 机器翻译 文本摘要 聊天机器人 语音识别 股票走势预测			

命名实体识别：我 爱 北京 天安门→主语 谓语 定语 宾语

词性标注：我 爱 北京 天安门→名词 动词 名词 名词

例10-1: 已知拓扑结构为同步多对多RNN（基本结构见图 RNN基本逻辑结构），输入层、隐含层（一层）、输出层的神经元均为一个，激活函数均为 ReLU， $W = [0.5, 0.1, 0.2]$ ， $H = [1]$ ， $V = [3]$ ， $S_0 = 0$ ， $\alpha = 0$ ， $\beta = 0$ ，对 $X = [[1,1,1], [2,2,2], [3,3,3]]$ 的输入序列，计算其输出序列 Y 。

解: $X_1 = [1, 1, 1]$ $X_2 = [2, 2, 2]$ $X_3 = [3, 3, 3]$

$$S_1 = f(W \bullet X_1^T + H \bullet S_0) = f(0.8) = 0.8$$

$$Y_1 = h(V \bullet S_1) = h(2.4) = 2.4$$

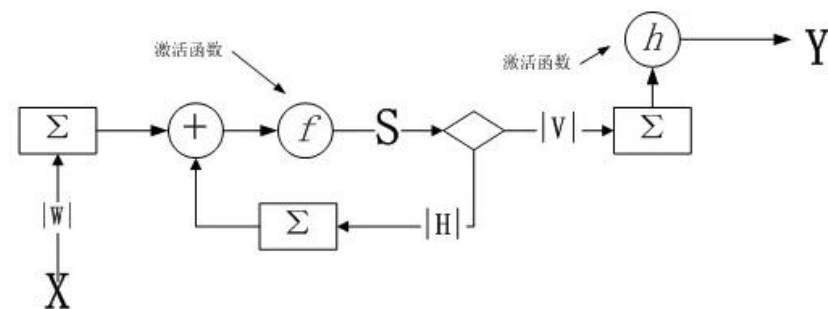
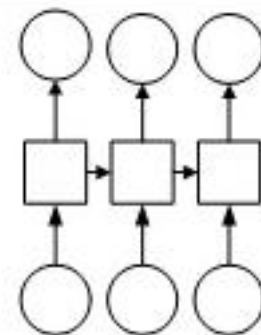
$$S_2 = f(W \bullet X_2^T + H \bullet S_1) = f(1.6 + 0.8) = 2.4$$

$$Y_2 = h(V \bullet S_2) = h(7.2) = 7.2$$

$$S_3 = f(W \bullet X_3^T + H \bullet S_2) = f(2.4 + 2.4) = 4.8$$

$$Y_3 = h(V \bullet S_3) = h(7.2) = 14.4$$

$$\therefore Y = [2.4, 7.2, 14.4]$$



$$Y_t = h(V \bullet S_t + \alpha)$$

$$S_t = f(W \bullet X_t + H \bullet S_{t-1} + \beta)$$

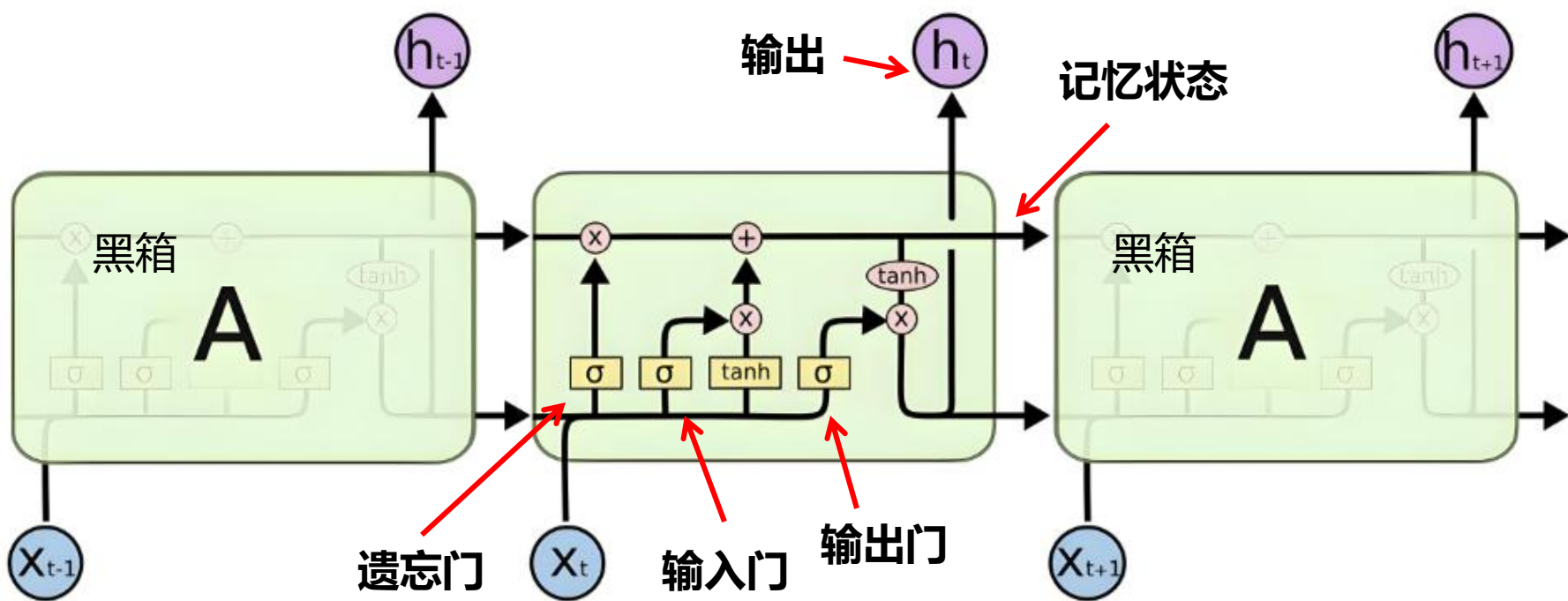


LSTM

LSTM的结构



RNN的问题：无法学习太长的序列，“很快忘记前面说过的话”



$\otimes$: 按位相乘 $\oplus$: 按位相加

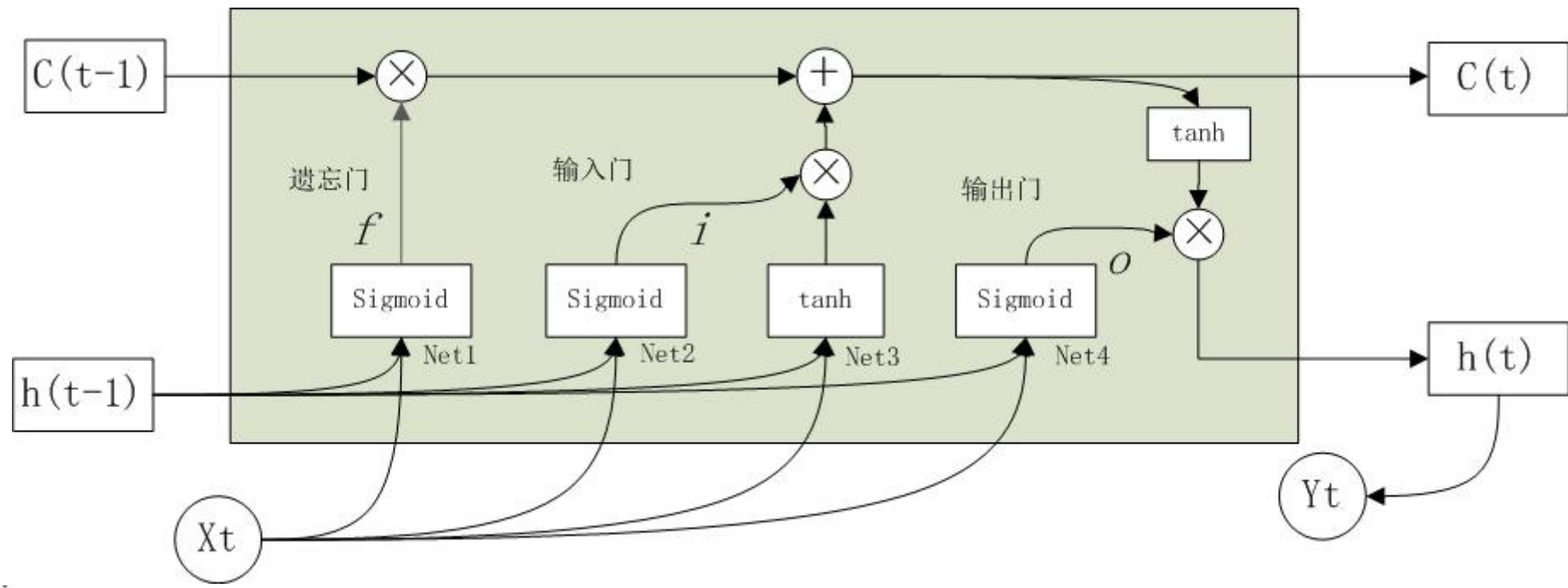
$$[1,2,3] \otimes [4,5,6] = [4,10,18]$$

$$[1,2,3] \oplus [4,5,6] = [5,7,9]$$

LSTM 按时间展开示意图

长短期记忆网络（Long Short-Term Memory, LSTM）：专门设计用于解决长期依赖的问题

LSTM的算法过程



网络参数:

$W_{h(1-4)}$

$W_{x(1-4)}$

b_{1-4}

	作用	计算方法
遗忘门	决定什么时候把以前的状态遗忘	$f_t = \text{sigmoid}(\text{Net1})$
输入门	决定什么时候加入新的状态	$i_t = \text{sigmoid}(\text{Net2})$
输出门	决定什么时候把状态和输入叠加输出	$o_t = \text{sigmoid}(\text{Net4})$
记忆状态	累积历史信息, 调控 h_t 输出内容	$C_t = f_t \otimes C_{t-1} + i_t \otimes \tanh(\text{Net3})$
隐式编码	与下一次的输入一起参与计算	$h_t = o_t \otimes \tanh(C_t)$

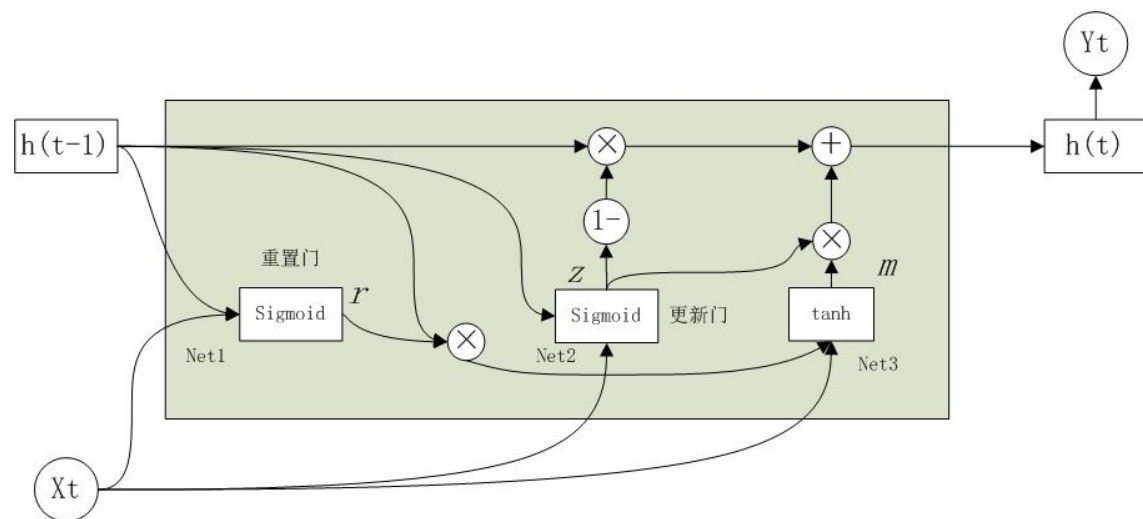
$\text{Net1} = W_{h1} \cdot h_{t-1} + W_{x1} \cdot X_t + b_1$
$\text{Net2} = W_{h2} \cdot h_{t-1} + W_{x2} \cdot X_t + b_2$
$\text{Net3} = W_{h3} \cdot h_{t-1} + W_{x3} \cdot X_t + b_3$
$\text{Net4} = W_{h4} \cdot h_{t-1} + W_{x4} \cdot X_t + b_4$

LSTM的局限及变种RNN

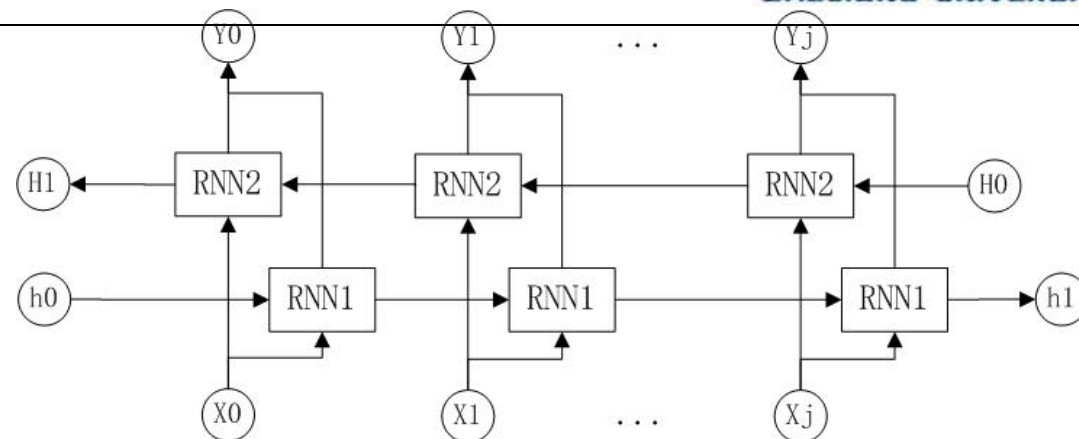


LSTM的局限

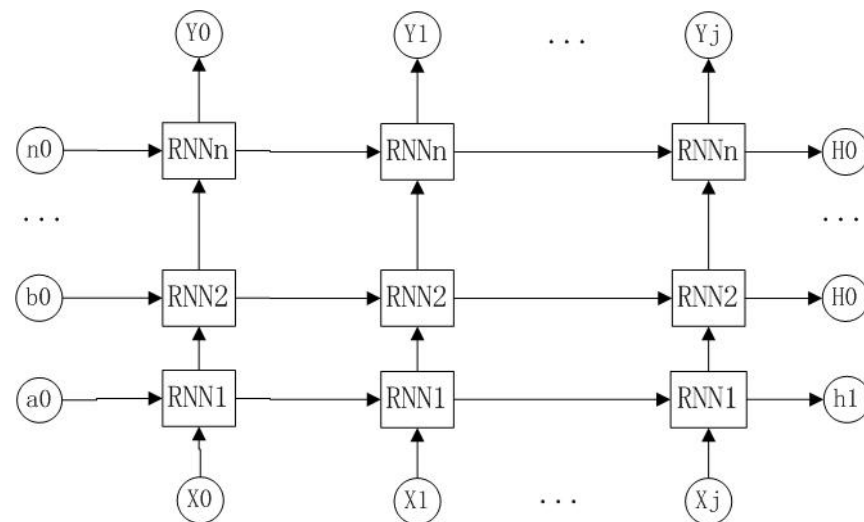
- 1、计算复杂
- 2、顺序处理，无法进行并行化计算
- 3、计算慢，实时响应难



门控循环单元 (gated recurrent unit, GRU)



双向RNN (Bi-directional RNN, Bi-RNN)

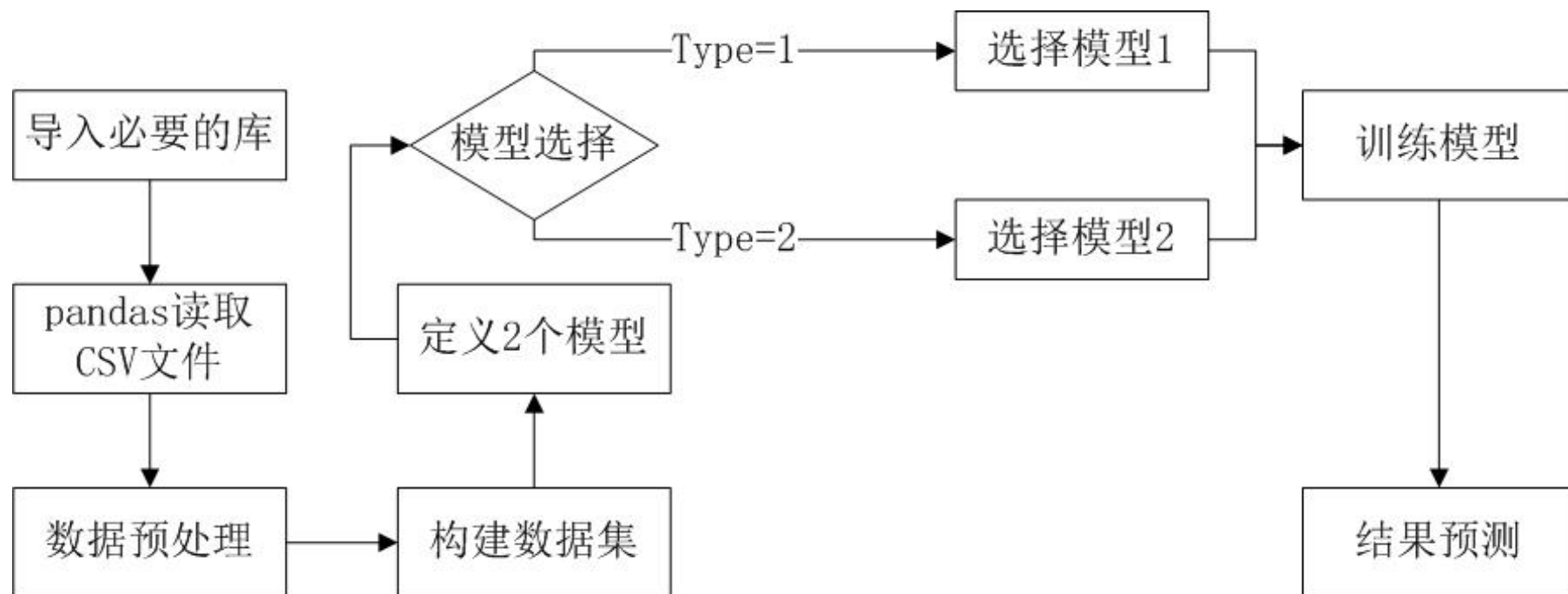


深度循环神经网络 (Deep Recurrent Neural Network)



代码案例：股票走势预测

程序流程图



模型参数设计及训练



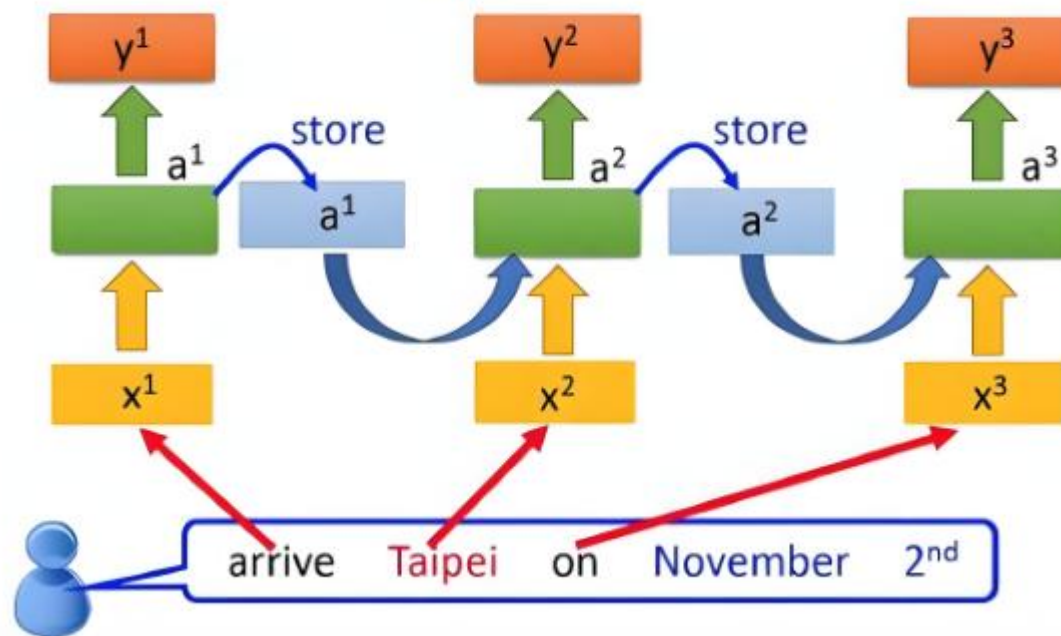
读取数据	pandas读取CSV数据: pd.read_csv() 数据清洗
构建数据集	选取特征值, 如X=PRE_CLOSE; Y=CLOSE
数据预处理	数据标准化 (归一化) 生成时间序列数据TimeseriesGenerator()
定义2个模型	设计2个或多个不同拓扑结构的LSTM网络, 可以比较不同的网络的预测效果。如深度、神经元个数、激活函数、dropout、优化器、损失函数、评价指标等
模型选择	根据输入的参数Type选择不同的模型
训练模型	指定训练轮次、批次大小、验证集等
结果预测	指定一序列输出预测结果

```
def model1():  
    model = Sequential()  
    model.add(LSTM(128, activation='relu', dropout=0.5, recurrent_dropout=0.5,  
                  input_shape=(n_input, 1), return_sequences=True))  
    model.add(LSTM(64, dropout=0.5, recurrent_dropout=0.5, activation='relu',  
                  return_sequences=True))  
    model.add(LSTM(1, dropout=0.5, recurrent_dropout=0.5, activation='relu',  
                  return_sequences=False))  
    model.add(Dense(1))  
    model.compile(optimizer='adam', loss='mse', metrics=['accuracy'])  
    return model  
def model2():  
    model = Sequential()  
    model.add(LSTM(50, activation='relu', input_shape=(n_input, 1)))  
    model.add(Dense(1))  
    model.compile(optimizer='adam', loss='mse', metrics=['accuracy'])  
    return model
```


Probability of
"arrive" in each slot

Probability of
"Taipei" in each slot

Probability of
"on" in each slot



THANKS